Carleton University | Department of Systems and Computer Engineering

Lab 3
Working with Variables, Visualizing Code Execution

Objectives

- To gain experience translating mathematical formulae into Python statements that use variables.
- To gain experience using the online Python Tutor to visualize the execution of the code.
- To gain experience using Python built-in functions.

Lab 3 covers lectures 3 and 4.

Getting Started

1. Download lab3.txt from Brightspace. Open this file in a text editor (for example, WordPad on a Windows computer or TextEdit if you are using a Mac). You can also open the file in Wing 101. If files with extension .txt are not listed in the file-chooser box, you will need to change the settings so that lab3.txt can be selected. If you are running Wing on a Windows computer, select the All Files (*.*) option. On a Mac, select the All Files (*) option.

   lab3.txt is a template in which you will type your solutions to the exercises. Please, do **not** change the layout of this file; that is, do not delete any lines other than the ones you are told to replace. Do not change the formatting; for example, do not use boldface or italicized text. You will be submitting your solutions as plain text.

2. Open Python Tutor in your web browser. The URL is:

   https://pythontutor.com/visualize.html

   One of the drop-down menus will be labelled, "inline primitives, don't nest objects [default]". We need to change this so that PyTutor's visualizations will be very similar to the textbook's memory model. Select "render all objects on the heap (Python/Java)" from this menu.

   Verify that PyTutor is now configured this way: "Write code in Python 3.6", "hide exited frames [default]", "render all objects on the heap (Python/Java)" and "draw pointers as arrows [default]".

Exercise 1: To convert a temperature from degrees Fahrenheit to degrees Kelvin, we subtract 32 from the Fahrenheit temperature, multiply that value by five-ninths, then add 273.15. For example, 70 degrees Fahrenheit is approximately 294.26 degrees Kelvin.

In PyTutor's editor window, type two assignment statements:

1. The first statement creates a new variable named `degrees_f` and assigns it (binds it to) the integer `70`.
2. The second statement converts the temperature referred to by `degrees_f` to the equivalent temperature in degrees Kelvin, using the formula presented above. This temperature should be assigned to a new variable named `degrees_k`.

Click the Visualize Execution button. Execute the code, one statement at a time, by clicking the Forward> button. Observe the memory model as the code is executed, step-by-step. Verify that your code converts 70 degrees Fahrenheit to (approximately) 294.26 degrees Kelvin. If necessary, correct your code and retest it.

Record your answers to these questions in lab3.txt:

(a) Write the Python statement that caused the variable `degrees_f` to appear in the memory model.

(b) What is the name of the frame containing `degrees_f`?

(c) What does the arrow that points from `degrees_f` to `70` represent?

(d) Write the Python statement that caused the variable `degrees_k` to appear in the memory model.

(e) The arrow in `degrees_k` points to an object. What is the type and value of this object?

Exercise 2: In some countries, a vehicle's fuel efficiency is measured in miles per gallon. In other countries, the efficiency is measured in litres per 100 km. For example, 32 miles per Imperial gallon is equivalent to approximately 8.83 litres per 100 km.

You are going to write Python statements that convert miles per gallon to litres per 100 km. One Imperial gallon is equal to approximately 4.54609 litres. One mile is equal to approximately 1.60934 km.

Hint: we recommend that, before you write any Python code, treat this as a math problem and derive a formula for converting miles per gallon to litres per 100 km, using the conversion factors listed in the previous paragraph. Verify that the units in the conversion factors cancel out properly.

In PyTutor, click Edit this code. The memory model from Exercise 1 will disappear. Delete the code in PyTutor's editor, then type four assignment statements:

1. The first statement creates a new variable named `mpg` and assigns it the value `32` (which represents 32 miles per gallon).

2. The second statement creates a new variable named `LITRES_PER_GALLON` and assigns it the value `4.54609`. (Note that the names of Python variables that have values that are supposed to remain constant are, by convention, usually written entirely in uppercase.)

3. The third statement creates a new variable named `KMS_PER_MILE` and assigns it the value `1.60934`.

4. The fourth statement converts the mileage referred to by `mpg` to the equivalent fuel consumption, measured in litres per 100 km, using the formula you derived earlier. This value should be assigned to a variable named `fuel_consumption`.

Click the Visualize Execution button. Execute the code, one statement at a time, and observe the memory model as the code is executed, step-by-step. Verify that your code converts 32 mpg to (approximately) 8.83 litres per 100 km. If necessary, correct your code and retest it.

When your code is correct, copy/paste the four assignment statements from PyTutor into lab3.txt.

Record your answers to these questions in lab3.txt, then save the file:

(a) What is the name of the first variable that appears in the memory model? What is the type and value of the object that it refers to?

(b) What is the name of the second variable that appears in the memory model? What is the type and value of the object that it refers to?

(c) What is the name of the third variable that appears in the memory model? What is the type and value of the object that it refers to?

(d) What is the name of the fourth variable that appears in the memory model? What is the type and value of the object that it refers to?

Exercise 3: Suppose you have some money (the *principal*) that is deposited in a bank account for a number of years and earns a fixed annual *rate* of interest. The interest is compounded *n* times per year.

The formula for determining the amount of money you will have is:

$$amount = principal\left(1 + \frac{rate}{n}\right)^{n \times time}$$

where:

- *amount* is the amount of money accumulated after *time* years, including interest.
- *principal* is the initial amount of money deposited in the account
- *rate* is the annual rate of interest, specified as a decimal; e.g, 5% is specified as 0.05, and not 5.
- *n* is the number of times the interest is compounded per year
- *time* is the number of years for which the principal is deposited.

For example, if $1,500 is deposited in an account paying an annual interest rate of 4.3%, compounded quarterly (4 times a year), the amount in the account after 6 years is:

$$amount = \$1500\left(1 + \frac{0.043}{4}\right)^{4 \times 6} \approx \$1938.84$$

In PyTutor, click Edit this code. Delete the code in PyTutor's editor, then:

1. Type four assignment statements that create new variables named `principal`, `rate`, `n` and `time`, and assign them the values `1500`, `0.043`, `4` and `6` respectively.

2. Type one assignment statement that calculates the amount of money in the account after `time` years, using the formula presented above. This value should be assigned to a new variable named `amount`. Your assignment statement must demonstrate the use of the Python built-in function `pow`.

Click the Visualize Execution button. Execute the code, one statement at a time, and observe the memory model as the code is executed, step-by-step. Verify that the accumulated amount of money is approximately `1938.84`.

When your code is correct, copy/paste the five assignment statements from PyTutor into lab3.txt.

Exercise 4: Here is an edited excerpt from *Practical Programming, 2nd Edition: An Introduction to Computer Science Using Python 3*:

In this example, the variable `score` appears on both sides of the assignment statement:

```
>>> score = 50
>>> score
50
>>> score = score + 20
>>> score
70
```

This is so common that Python provides a shorthand notation for this operation:

```
>>> score = 50
>>> score
50
>>> score += 20
>>> score
70
```

An augmented assignment combines an assignment statement with an operator to make the statement more concise. An augmented assignment statement is executed as follows:

1. Evaluate the expression on the right of the `=` sign to produce a value.
2. Apply the operator attached to the `=` sign to the variable on the left of the `=` and the value that was produced in step 1. This produces another value. Store the binding to that value in the variable on the left of the `=`.

Note that the operator is applied *after* the expression on the right is evaluated:

```
>>> d = 2
>>> d *= 3 + 4
>>> d
14
```

All of the binary operators, `+`, `-`, `*`, `/`, `//`, `%` and `**`, have shorthand versions. For example, we can square a number by multiplying it by itself:

```
>>> number = 10
>>> number *= number
>>> number
100
```

This code is equivalent to:

```
>>> number = 10
>>> number = number * number
>>> number
100
```

This table contains a summary of the augmented assignment operators:

Symbol	Example	Result
`+=`	`x = 7` `x += 2`	x refers to 9
`-=`	`x = 7` `x -= 2`	x refers to 5
`*=`	`x = 7` `x *= 2`	x refers to 14
`/=`	`x = 7` `x /= 2`	x refers to 3.5
`//=`	`x = 7` `x //= 2`	x refers to 3
`%=`	`x = 7` `x %= 2`	x refers to 1
`**=`	`x = 7` `x **= 2`	x refers to 49

Without using the Python shell or PyTutor to assist you, predict the value that `x` will refer to after Python executes these two assignment statements:

```
x = 5
x *= x + x
```

In PyTutor, click Edit this code. Delete the code in PyTutor's editor, then type the two statements.

Click the Visualize Execution button. Execute the code, one statement at a time, and observe the memory model as the code is executed, step-by-step. Were your predictions correct?

In lab3.txt, write a short step-by-step explanation of what happens when Python executes the statement, `x *= x + x`, when `x` refers to `5`.

Wrap Up

You **do not** need to submit lab3.txt to Brightspace.

Once you finish your lab, remember to complete the **Lab 3 Quiz** on Brightspace to get marks for your lab work.